

Infrastructure Scripting

(Terraform HCL)

Fundamentals of System Security
(COMSM0122)

Learning Resources

2017 IEEE/ACM 39th IEEE International Conference on Software Engineering Companion

DevOps: Introducing Infrastructure-as-Code

Matej Artač², Tadej Borovšak², Elisabetta Di Nitto¹, Michele Guerriero¹, Damian Andrew Tamburri¹

¹DEEPSE group - DEIB - Politecnico di Milano, Milano, Italy

²XLAB - Ljubljana, Slovenia

matej.artac@xlab.si, tadej.borovsak@xlab.si, elisabetta.dinitto@polimi.it,

michele.guerriero@polimi.it, damianandrew.tamburri@polimi.it

Abstract—DevOps entails a series of software engineering tactics aimed at shortening the actionable operation of software design changes. One of these tactics is to harness infrastructure-as-code, that is, writing a blueprint that contains deployment specifications ready for orchestration in the cloud. This abstract briefly discusses all necessary elements and abstractions in writing and maintaining that blueprint, revolving around a key standard for its expression, namely, the OASIS “Topology and Orchestration Specification for Cloud Applications” (TOSCA) industrial standard adopted by as many as 60+ big industrial players worldwide.

Keywords—DevOps; Infrastructure-as-Code; TOSCA;

I. INTRODUCTION

The current IT market is increasingly dominated by the “need for speed”. This need is reflected in the emerging use of agile and lean techniques which shorten the software development cycle and also intermix software development activities with IT operations [1]. This trend of using software engineering tactics that reduce the space, time and efforts between software development and operations as well as the technical and organizational distance between these two types of software teams is known as DevOps [1]. As part of the DevOps menu, many practices entail re-using standard tools from software development (e.g., code-versioning, code-revision management, etc.) to manage what is known as *infrastructure-as-code* (IaC) [2]. IaC promotes managing knowledge and experience of plethora of subsystems as a single commonly available source of truth instead of traditionally reserving it for system administrators. This briefing introduces all essential technologies involved in supporting infrastructure-as-code, starting from the industrial standard for IaC, OASIS TOSCA, which stands for “Topology and Orchestration Specification for Cloud Applications” [3] and elaborating more on connected technologies, such as deployment blueprints or TOSCA-ready orchestration engines. TOSCA is state-of-the-art industrial practice language for automated deployment of technology independent and multi-cloud compliant applications, supported by players such as HP, CISCO Systems, Cloudera, Huawei, IBM [3].

The following contains a brief overview of DevOps, addressing TOSCA and IaC together.

II. DEVOPS IN PILLS

In essence the core idea behind DevOps is fairly simple: over long years of “siloed” [4] software development (on

one side) and software operations (on the other), a series of distances in technical, socio-technical, and organizational practices and tools has been flourishing between Dev- and -Ops teams [1]. These distances make the fluent and coherent communication, collaboration and issue resolution between the two silos practically difficult if not impossible. DevOps entails a series of socio-technical and organizational practices and tools that address and tackle these distances one by one until a *frictionless*, fluent, and resilient organization is distilled. Of the several socio-technical practices under study we mention, most prominently, the organizational blending of Dev- and -Ops teams under the same working unit with consequent shifting of operational concerns (think of, for example, infrastructure design, dimensioning, cost-optimization [5]) to be addressed by the entire team, including developers. Other similar socio-technical and organizational practices lean towards the shared use of “DevOps” tools [6], i.e., tools that promote the use of the same knowledge-base (think of an issue-tracking mechanism) for both development and operation such that there shall be no misunderstanding or knowledge gap between developer and operator knowledge at all.

III. INTRODUCING INFRASTRUCTURE-AS-CODE

Infrastructure design is the software lifecycle phase that defines and configures the software infrastructure needs for that software, e.g., the AWS cloud, the number and type of VMs required, etc. Infrastructure designs typically entail many installation and configuration scripts needed to, among others: (a) instantiate and link the required machines (either physical or virtual) for the software to run; (b) install and configure the required software and middleware for those VMs; (c) instantiate and run the needed ancillary services for the software to be operated. In the scope of these designs, DevOps promotes the use of a typical software development notion, i.e., “source code”, for infrastructure designs as well, such that the entire set of scripts, automation and configuration code, models, required dependencies and operational configuration parameters can be expressed using the same standard language, much like you would use Java and ancillary Java libraries to put together your own software application - this practice is *infrastructure-as-code*. The purpose behind this proposal is to re-use successful and common software development practices to speed up software operations: think of using infrastructure code versioning for the purpose of de-bugging and back-tracking/version control or using infrastructure design



O'REILLY®

Infrastructure as Code

Dynamic Systems for the Cloud Age

Second Edition



Kief Morris

What is Terraform?

- Terraform is an **open-source infrastructure as code (IaC)** software tool created by HashiCorp

Define Infrastructure

Use Terraform's configuration language to describe the desired infrastructure.

Manage Infrastructure

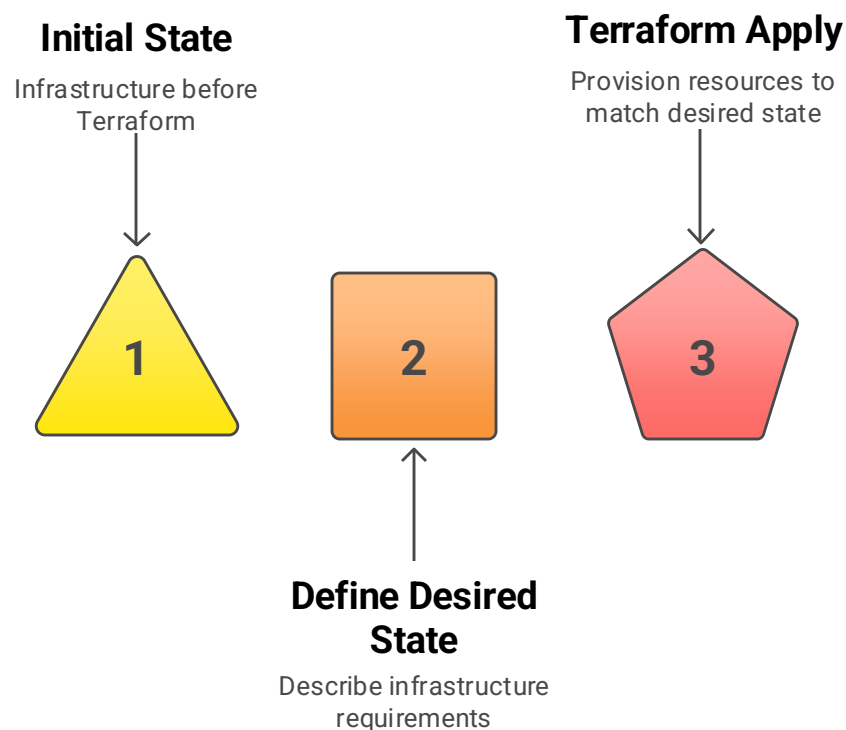
Apply Terraform configurations to create and manage infrastructure resources.

Collaborate Securely

Work with team members in a stable environment to manage infrastructure.

What is Terraform?

- Terraform uses a declarative configuration language, meaning that it allows users to describe the desired state of the cloud resources.



Terraform Configuration Process

What is Terraform?

- Example: Two similar configuration files, where a user wants 3 servers to be deployed:

AWS Terraform code

```
resource "aws_instance" "web-servers" {  
  ami      = "ami-abcd"  
  instance_type = "t3.micro"  
  count    = 3  
}
```

1

Proxmox Terraform code

```
resource "proxmox_vm_qemu" "web_servers" {  
  count      = 3  
  name       = "web-server-${count.index}"  
  target_node = "proxmox-node1"  
  vmid       = 100 + count.index  
  
  clone      = "template-ubuntu"  
  cores      = 1  
  memory     = 1024  
  sockets    = 1  
}
```

1

```
network {  
  model = "virtio"  
  bridge = "vmbr0"  
}  
  
disk {  
  size      = "10G"  
  type      = "scsi"  
  storage   = "local-lvm"  
}  
  
os_type = "cloud-init"  
ipconfig0 = "ip=dhcp"  
}
```

2

Terraform will **analyse** the state of all the resources it manages, and **execute** the required actions and API calls to **set** the cloud resources to the **desired state**.

Why do we need Terraform?

Terraform provides clarity with respect to infrastructure and cloud configuration. It lets users:

- Spin up the same configuration for testing and staging environments.
- Synchronise changes, making sure that testing environments are similar to production.
- Track changes throughout deployments for review processes.
- Terraform also makes infrastructure knowledge transfer very easy, and improves security by limiting access for manual actions.

Quiz

Which of the following are true about Terraform?

- A. Terraform doesn't let users create the same configurations for testing and staging environments.
- B. Terraform lets users specify the changes they want to make to their cloud resources.
- C. Terraform lets users manage their resources on multiple cloud platforms.
- D. Users can commit configurations to version control to safely collaborate on infrastructure.
- E. Terraform limits manual changes, thereby acting as an additional security measure

Providers, Resources and Terraform State

Provider:

- Terraform interacts with over 1000 cloud platforms and services using their APIs. These interactions are made possible by Terraform plugins, which are called **providers**
- **Providers** are specified in the Terraform configuration code, which specifies the services it needs to interact with
- Once specified, each **provider** shows all the different resources and **data types** available for use.

Providers, Resources and Terraform State

Resources:

- **Resources** are the most important elements in the Terraform language. They're defined as blocks of code, called **resource blocks**.
- Each **resource block** can define one or more infrastructure objects. For example, compute instances or private networks. Resources can be composed of objects from different providers as well.

Providers, Resources and Terraform State

Terraform State:

- Terraform records and logs information about all the resources it creates and manages in a file, called the state file.
- This state file acts as a source of truth for Terraform and maps the objects in its configuration to the real world (remote instances and infrastructure).
- It uses this state file and determines all the changes required to align the infrastructure with the declared configuration.

Quiz

Which of the following is **TRUE**?

- A. The state file describes the list of changes needed to align the infrastructure with the declared configuration.
- B. Resource blocks** can define one or more infrastructure objects, from any number of providers.
- C. Terraform plugins called **providers** specify the services it needs to interact with.

Install Terraform

Terraform is written in `Go` programming language and does not have any special dependencies

- It can be installed on any **Linux**, **Windows** or **Mac** machine and can be used in CI/CD to apply **infrastructure changes** automatically.

<https://developer.hashicorp.com/terraform/downloads>

Install Terraform

How to download the Terraform binary on a Linux machine

```
wget https://releases.hashicorp.com/terraform/1.3.4/terraform_1.3.4_linux_amd64.zip  
unzip terraform_1.3.4_linux_amd64.zip  
chmod +x terraform
```

Check the installed version of Terraform:

```
terraform --version
```

Example: Create and Initialise an Infrastructure

main.tf

```
resource "aws_instance" "test" {  
  ami = "ami-0ff8a91507f77f867"  
  instance_type = "t2.micro"  
}
```

Declares a new AWS EC2 instance resource named **test**. This is how Terraform knows you're provisioning a virtual machine in AWS.

Specifies the Amazon Machine Image (AMI) ID to use. This AMI is a pre-configured OS image (like Ubuntu, etc.) that the instance will boot from. This particular AMI ID is region-specific (us-east-1 region).

Defines the size and performance characteristics of the instance. `t2.micro` is a low-cost, general-purpose instance type with 1 vCPU and 1 GiB of memory—often used for testing or small workloads.

Example: Create and Initialise an Infrastructure

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

terraform init

- Initialise the working directory by setting up the **.terraform** folder where it stores internal data and configuration.
- Based on the code, it downloads the required providers (**aws**), and fetches the appropriate plugins from the Terraform Registry or other sources.
- Install backend configuration (if defined). For example, if using remote state (e.g., S3, Consul), it configures that backend.
- Prepare for future commands by ensuring that **terraform plan**, **apply**, and others will work smoothly by validating the setup.

Example: Create and Initialise an Infrastructure

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

terraform init

- Check version compatibility between **modules**, **providers** and **code**.
- Download all used plugins and modules to the **.terraform** module

File Structure

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

```
terraform init
```



- The `.terraform` module should usually be excluded from version control.
- Use `terraform init` or `terraform init -upgrade` to update all components to their **latest** versions.

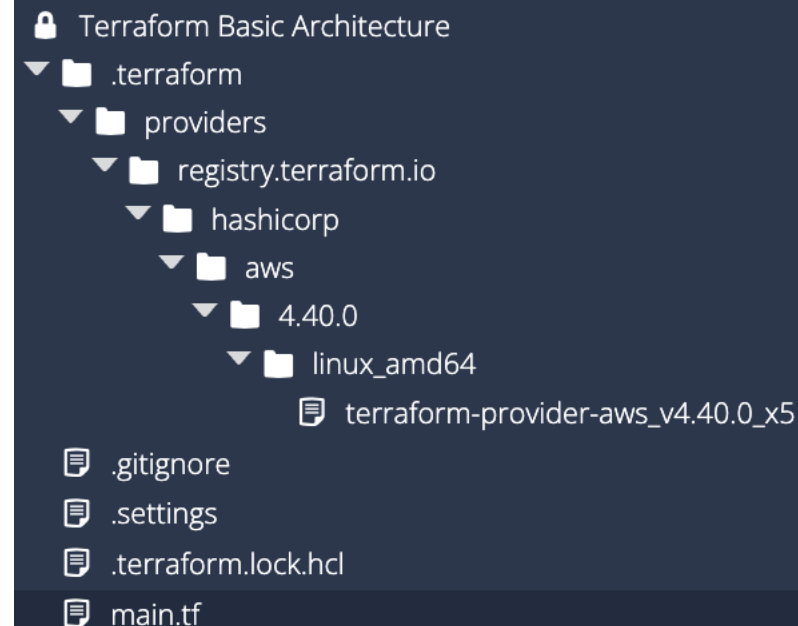
Locks

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

```
terraform init
```



- The **.terraform.lock.hcl** locks the dependencies to the exact versions (hashes) for components in the configuration.
- Terraform will always reselect versions of components it uses from the lock file unless **-upgrade** is specified for the **init** command.
- If a particular **provider** has no existing recorded selection, then the **latest** version that matches the given version constraint will be selected and added in the lock file

Example: Create and Initialise an Infrastructure

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

terraform plan

Preview what Terraform will do based on your code.

- Read your configuration files (**.tf files**).
- Compare your desired infrastructure (from code) with the current state (from the state file or remote backend).
- Generate an execution plan showing:
 1. What resources will be created, changed, or destroyed?
 2. Any dependencies or ordering.
 3. Whether changes are safe or potentially destructive.

Example: Create and Initialise an Infrastructure

main.tf

```
resource "aws_instance" "test" {  
    ami = "ami-0ff8a91507f77f867"  
    instance_type = "t2.micro"  
}
```

Commands:

terraform apply

Actually create or modify infrastructure as defined.

- Launch a new EC2 virtual machine in AWS.
- Use the specified AMI as the base image.
- Allocate a **t2.micro** instance type.
- Assign it a default name **test** (internally in Terraform).

State File

Terraform State:

- The Terraform state file (`terraform.tfstate`) is created the first time you run `terraform apply`—after you've already run `terraform init` and `terraform plan`.
- It stores:
 1. Resource IDs (like EC2 instance IDs)
 2. Metadata (like IP addresses, disk sizes)
 3. Dependencies between resources

Terraform Ecosystem

Command	Creates State File?	Description
<code>terraform init</code>	No	Initializes the project and downloads providers
<code>terraform plan</code>	No	Previews changes but doesn't touch infrastructure
<code>terraform apply</code>	Yes	Applies changes and creates/updates the state file

Terraform Constructs and HCL Syntax

We will only go over the syntax for the fundamental building blocks of a Terraform configuration. For more information Refer to:

<https://developer.hashicorp.com/terraform/language>

Terraform Constructs and HCL Syntax

- The **native, low-level** syntax of the Terraform language is called **HCL** (HashiCorp Configuration Language).
- The full syntax definition:
<https://github.com/hashicorp/hcl/blob/main/hclsyntax/spec.md>

Terraform Constructs and HCL Syntax

Example:

```
io_mode = "async"
service "http" "web_proxy" {
  listen_addr = "127.0.0.1:8080" process
  "main" {
    command = ["/usr/local/bin/awesome-app", "server"]
  }
  process "mgmt" {
    command = ["/usr/local/bin/awesome-app", "mgmt"]
  }
}
```

- This configuration sets up a local HTTP service called `web_proxy` that listens on port 8080 and runs two separate processes:
 1. One for serving the application (server);
 2. One for managing it (mgmt)
- It uses asynchronous I/O to handle operations efficiently.

Key Elements

HCL is built around two primary concepts:
attributes and **blocks**.

■ Attributes

An attribute assigns a value to a particular name

```
io_mode = "async"
```

Key Elements

HCL is built around two primary concepts:
attributes and **blocks**.

■ Block

A block is a container for content.

```
resource "aws_instance" "example" {  
  ami = "abc123"  
  network_interface {  
    # ...  
  }  
}
```

Each block has a **type**.

Depending on the **type**, each block requires a certain number of labels to follow.

The body of each block is delimited by { and }

In this example, the block type is **resource**.

And a block of type **resource** expects 2 labels to follow ("**aws_instance**" and "**example**")

Key Elements

HCL is built around two primary concepts:
attributes and **blocks**.

■ Identifiers

Attribute/Argument names, block type names, and the names of most Terraform-specific constructs like resources, input variables, etc. are all called identifiers.

Identifiers can contain letters, digits, underscores (`_`), and hyphens (`-`). The first character of an identifier must not be a digit, to avoid ambiguity with literal numbers.

Providers

We define providers in Terraform with the **provider** block.

```
provider "aws" {  
    region = "us-east-1"  
}
```

- Terraform plugins are called providers
- Providers are used to interact with Infrastructure as a service (IaaS) or Platform as a service (PaaS) APIs.
- Providers also define which **resources** are available to create, update or delete.

Resources

A Terraform **resource** is any infrastructure component that is managed by **providers**.

Example:

```
resource "aws_instance" "xx" {  
  ami = "ami-005e54dee72cc1d00" # us-west-2  
  instance_type = "t2.micro"  
  
  network_interface {  
    network_interface_id = aws_network_interface.xx.id  
    device_index = 0  
  }  
  credit_specification {  
    cpu_credits = "unlimited"  
  }  
}
```

resource "aws_instance" "xx"
Declares an EC2 instance resource named xx.

Specifies the Amazon Machine Image (AMI) ID to use

Defines the size and performance of the instance

Resources

A Terraform **resource** is any infrastructure component that is managed by **providers**.

Example:

```
resource "aws_instance" "xx" {  
  ami = "ami-005e54dee72cc1d00" # us-west-2  
  instance_type = "t2.micro"
```

```
  network_interface {  
    network_interface_id = aws_network_interface.xx.id  
    device_index = 0  
  }  
  credit_specification {  
    cpu_credits = "unlimited"  
  }  
}
```

- Attaches a **predefined network interface** to the instance
- `aws_network_interface.xx.id` references another resource (not shown here) that defines the network interface

- `device_index = 0` means this is the primary network interface

Resources

A Terraform **resource** is any infrastructure component that is managed by **providers**.

Example:

```
resource "aws_instance" "xx" {  
  ami = "ami-005e54dee72cc1d00" # us-west-2  
  instance_type = "t2.micro"  
  
  network_interface {  
    network_interface_id = aws_network_interface.xx.id  
    device_index = 0  
  }  
  credit_specification {  
    cpu_credits = "unlimited"  
  }  
}
```

A reference resource that this code depends on.

Enables **unlimited CPU credits** for burstable performance

Resources:

Referenced aws_network_interface:

```
resource "aws_network_interface" "xx" {  
  subnet_id      = "aws_subnet.example.id"  
  private_ips    = ["10.0.1.100"]  
  security_groups = ["aws_security_group.example.id"]  
  
  tags = {  
    Name = "custom-network-interface"  
  }  
}
```

A reference resource that this code depends on.

Resources :

Referenced aws_subnet:

```
resource "aws_subnet" "example" {  
    vpc_id          = "vpc-abc123"  
    cidr_block      = "10.0.1.0/24"  
    availability_zone = "us-west-2a"  
    map_public_ip_on_launch = true  
  
    tags = {  
        Name = "example-subnet"  
    }  
}
```

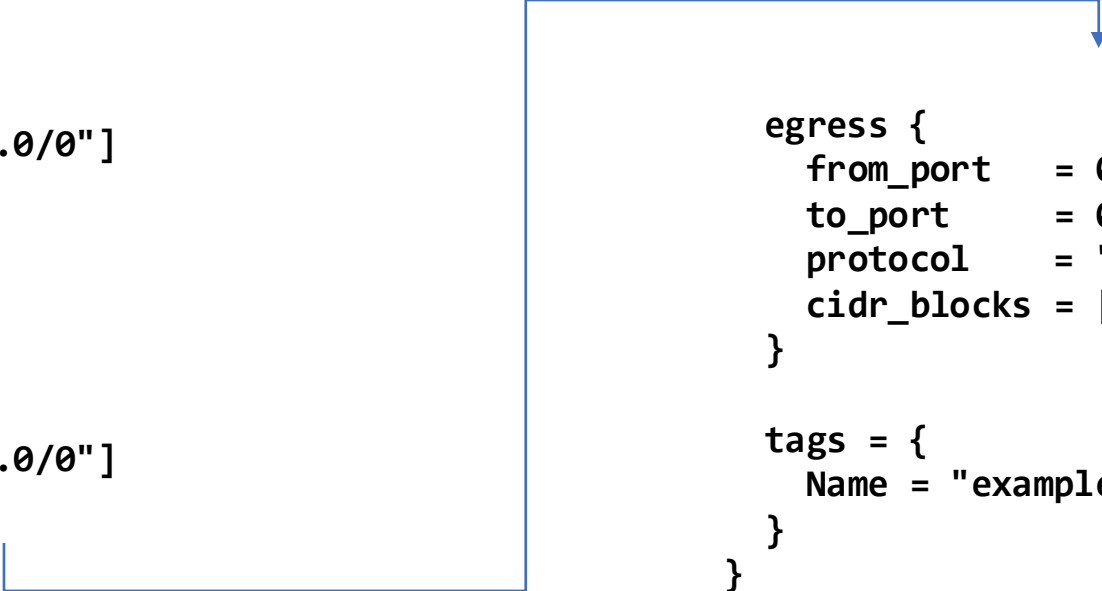
Resource:

Referenced Security Group Resource:

```
resource "aws_security_group" "example" {  
  name          = "example-sg"  
  description   = "Allow HTTP and SSH"  
  vpc_id        = "vpc-abc123" # Same VPC as subnet
```

```
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

```
  ingress {  
    from_port = 80  
    to_port   = 80  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```



```
  egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  tags = {  
    Name = "example-security-group"  
  }  
}
```

Resource:

Ingress = Incoming Traffic

Ingress rules define what traffic is allowed into your resource (like an EC2 instance).

Example: Allowing SSH (port 22) from your IP so you can log into the server.

```
ingress {  
  from_port    = 22  
  to_port      = 22  
  protocol     = "tcp"  
  cidr_blocks  = ["203.0.113.0/24"]  
}
```

This rule allows TCP traffic on port 22 from the IP range **203.0.113.0/24**.

Resource:

Egress = Outgoing Traffic

Egress rules define what traffic is allowed out of your resource.

Example: Allowing all outbound traffic so your server can access the internet.

```
egress {  
    from_port    = 0  
    to_port      = 0  
    protocol     = "-1"          # -1 means all protocols  
    cidr_blocks  = ["0.0.0.0/0"] # All destinations  
}
```

This rule allows all outbound traffic to any destination.

Resources – Change Management

Any changes in attributes either trigger a change in the resource or the recreation of the resource if an in-place change is not possible

How can an attribute be updated without triggering changes?

- The **lifecycle** attribute defines the behaviour of a resource when an attribute changes.

Resources – Change Management

■ Example.

```
lifecycle {  
  ignore_changes = [  
    ami,  
    key_name,  
    user_data,  
  ]  
}
```

1. **ami**: If the AMI ID changes in your .tf file, Terraform will **not** recreate the instance.
2. **key_name**: Changes to the SSH key used for access will be ignored.
3. **user_data**: Updates to the startup script won't trigger a replacement.

Input Variables

- You can define input variables to serve as Terraform configuration parameters.
By convention, input variables are defined by using a variable block, within a file named **variables.tf**.

Input Variables

Example: `variables.tf`

```
variable environment {  
    default = "production"  
    type = string  
    description = "Environment type"  
}  
  
variable region {  
    default = "us-east-1"  
    type = string  
    description = "Region for resources"  
}  
  
variable xx {  
    type = number  
    description = "xx number"  
}
```

- In this example, 2 variables, **environment** and **region**, have been assigned a default value.
- Adding a **default value** makes that variable **optional**.

Input Variables

Variable types supported in Terraform

- **string** - string values
- **number** - numeric value (fractional and whole)
- **bool** - Boolean
- **list** - a sequence of values ["us-west-1a", "us-west-1c"]
- **map** - a group of values identified by named labels
{name = "Mabel", age = 52}
- **null** - a value that represents absence or omission

Input Variables

How do we set or assign values to variables?

- Individually, with the -var command line option.

Example:

```
terraform apply -var="region=us-east-2"
```

Input Variables

How do we set or assign values to variables?

- In a Variable Definitions File (.tfvars)

Example of a .tfvars file:

```
foo = 2  
environment = "staging"  
region = "us-west-1"
```

A pointer to the file is then specified on the command line

```
terraform apply -var-file="vars.tfvars"
```

Input Variables

How do we set or assign values to variables?

- Using environment variables

As a fallback measure, Terraform will **always** search its **own process environment** for variables named in the following format: **TF_VAR_<variable_name>**

Output Variables

If we compare the Terraform language to other programming languages, output variables would be like the return values of a function.

- Returned values can be hashed/encrypted passwords or server Ips
- Exposes infrastructure information for other Terraform configs to use
- Output variables are defined by using the output block

```
output "ip" {  
    value = "${aws_instance.foo.ip_address}"  
}
```

Referencing Variables

Any defined variable can be used in the resources attributes using `var.<variable_name>`

Example:

```
resource "aws_instance" "foo" {  
    ami = var.ami  
    instance_type = var.type  
}
```

Referencing Variables

■ Interpolation

Referencing variables and other named values using syntax - `${}`

Allows variable references and to work with more complex cases

Example:

```
resource "aws_instance" "foo" {  
  ami = var.ami  
  instance_type = var.type  
  tags {  
    "Name" = "foo-${var.environment}"  
  }  
  subnet_id = "${var.env == "production" ? var.prod_subnet : var.dev_subnet}"  
}
```


Functions

HCL includes a number of built-in functions that can be called from within expressions to transform and combine values.

- The syntax for a function call is `<FUNCTION_NAME>(<ARG_1>, ...)`

Example:

```
max(5, 12, 9)
```

- Terraform does not support user-defined functions. You can only use built-in functions in Terraform.

Functions

Terraform's built-in functions can be classified into the following groups:

- Numeric functions - to manipulate numbers
- String functions - to manipulate strings
- Collection functions - to manipulate arrays and other collections
- Encoding functions - support base64, json, csv, yaml encoding and decoding
- Filesystem functions - to manipulate filesystem paths
- Date and Time functions
- Hash and Crypto functions
- IP Network functions - to manipulate IP addresses
- Type Conversion functions

<https://developer.hashicorp.com/terraform/language/functions>

Workshop 3: Proxmox Terraform Provider

- Terraform schema for Proxmox

<https://github.com/Telmate/terraform-provider-proxmox>

https://registry.terraform.io/providers/Telmate/proxmox/latest/docs/guides/cloud_init